

Surrogate Modeling in Statistical Learning:

A Deep Dive into Gaussian Processes

Naman Shukla (BT24CS032)

Indian Institute of Technology Madras

Research Interns under the guidance of:

Prof. Neelesh Shankar Upadhye

Department of Mathematics

June 2026

Key Insight: We don't need the weights w to define the process!

By defining the covariance function (kernel) directly:

$$k(x, x') = \phi(x)^T \Sigma_p \phi(x')$$

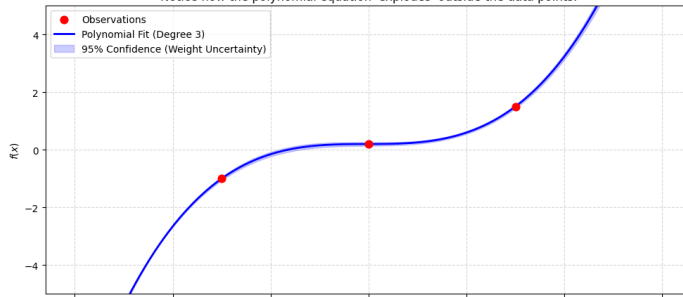
- ▶ **The Kernel Trick:** We bypass the weight space and compute inner products in feature space.

$$k(x, x') = \psi(x) \cdot \psi(x')$$

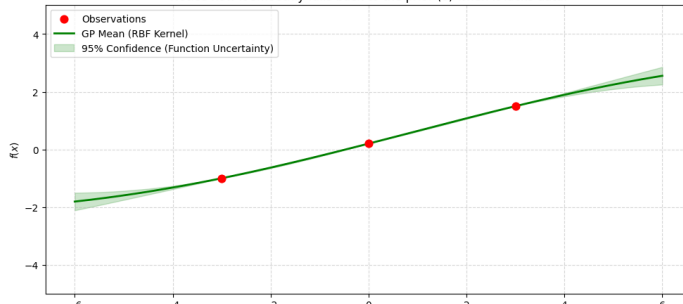
where, $\psi(x) = \Sigma_p^{1/2} \phi(x)$

- ▶ **Infinite Dimensions:** Kernels allow us to handle infinite-dimensional feature maps.
- ▶ We now focus on the properties of functions rather than individual parameters.

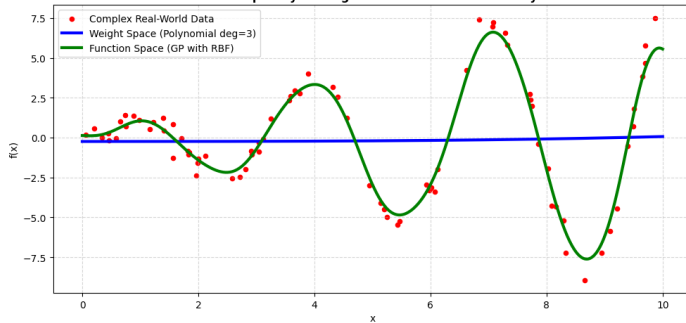
Weight Space View (Parametric)
Notice how the polynomial equation 'explodes' outside the data points.



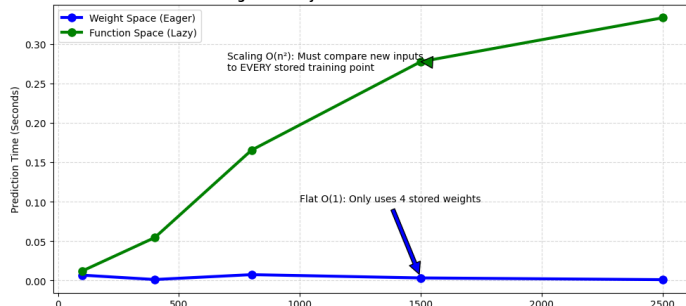
Function Space View (Non-Parametric)
Notice how it smoothly falls back to the prior (0) when it lacks data.



1. Complexity Scaling: Fixed vs. Infinite Flexibility



2. Eager vs. Lazy: Time to Predict 1000 New Points



Instead of modelling weights:

$$f \sim GP(m, k)$$

- ▶ Directly inference over functions
- ▶ Infinite-dimensional perspective
- ▶ Covariance describes relationships

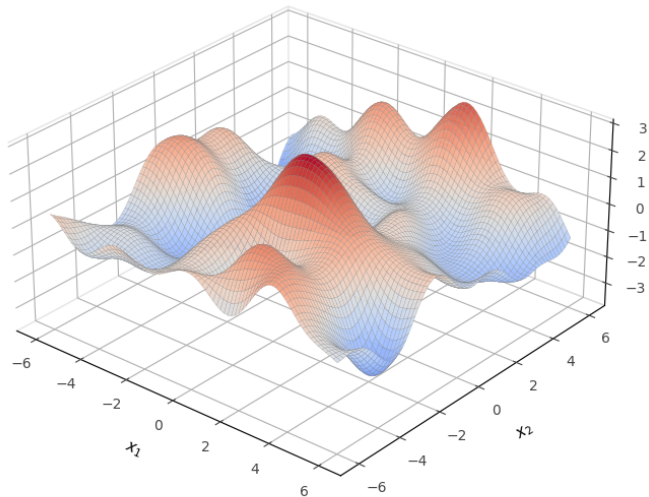
Definition: A Gaussian process is a collection of random variables, any finite number of which have (consistent) Gaussian distributions.

$$\mathbf{f} = (f_1, \dots, f_n)^T \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma}), \quad \text{indexes } i = 1, \dots, n$$

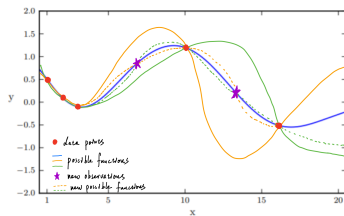
A Gaussian process is fully specified by a mean function $m(x)$ and covariance function $k(x, x')$ where it forms a PSD matrix called Gram Matrix:

$$f(x) \sim \mathcal{GP}(m(x), k(x, x')), \quad \text{indexes: } x$$

Function drawn at random from a Gaussian Process
with Gaussian covariance



Why Gaussian Processes?



- Distribution over functions
- Bayesian learning framework
- Non-parametric model
- Works well on small dataset

Instead of learning a single function, GPs learn a distribution over possible functions.

Consistency and Kolmogorov Extension

- ▶ A Gaussian Process is a collection of random variables such that every finite subset is jointly Gaussian.
- ▶ The finite-dimensional distributions must be mutually consistent.
- ▶ This is known as the **marginalization property**.

If

$$(y_1, y_2) \sim \mathcal{N}(\mu, \Sigma)$$

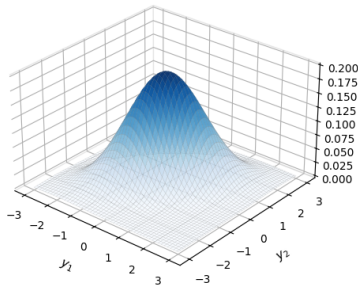
then marginalizing over y_2 gives

$$y_1 \sim \mathcal{N}(\mu_1, \Sigma_{11})$$

Kolmogorov Extension Theorem states that any consistent family of finite-dimensional distributions defines a valid stochastic process.

Consistency and Kolmogorov Extension

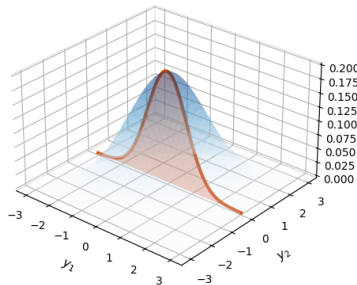
Joint Distribution
 $p(y_1, y_2)$



- ① **Joint** - the full hill
 $p(y_1, y_2)$

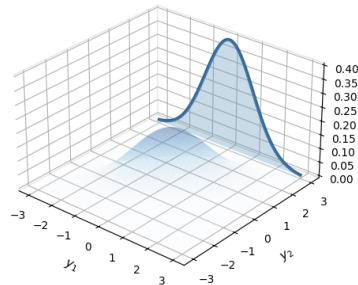
Marginalization: Integrating Out y_2

Slice at $y_2 = 0$
 $p(y_1 | y_2 = 0)$



- ② **Conditional** - cut the hill
 $p(y_1 | y_2 = c)$

Marginal $p(y_1)$
 $= \int p(y_1, y_2) dy_2$



- ③ **Marginal** - flatten the hill
 $p(y_1) = \int p(y_1, y_2) dy_2$

Noise-Free Prediction

$$f_*|X, X_*, f \sim \mathcal{N}(K(X_*, X)K(X, X)^{-1}f, K(X_*, X_*) - K(X_*, X)K(X, X)^{-1}K(X, X_*))$$

Prediction with Gaussian Noise

$$y = f(x) + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma_n^2)$$

$$f_*|X, y, X_* \sim \mathcal{N}(\bar{f}_*, \text{cov}(f_*))$$

$$f_* = K(X_*, X) [K(X, X) + \sigma_n^2 I]^{-1} y$$

$$\text{cov}(f_*) = K(X_*, X_*) - K(X_*, X) [K(X, X) + \sigma_n^2 I]^{-1} K(X, X_*)$$

Key Difference:

$$K(X, X) \rightarrow K(X, X) + \sigma_n^2 I$$

$$\log p(y | X) = \underbrace{-\frac{1}{2}y^T(K + \sigma_n^2 I)^{-1}y}_{\text{data fit}} \underbrace{-\frac{1}{2}\log|K + \sigma_n^2 I|}_{\text{complexity penalty}} \underbrace{-\frac{n}{2}\log(2\pi)}_{\text{normalisation}}$$

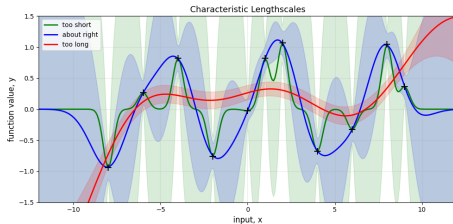
- **Hyperparameter tuning:** Kernel hyperparameters θ (e.g. length-scale ℓ , signal variance σ_f^2) and noise σ_n^2 are optimised by maximising the log marginal likelihood:

$$\hat{\theta} = \arg \max_{\theta} \log p(y | X, \theta)$$

- Gradients $\nabla_{\theta} \log p(y | X, \theta)$ are tractable, enabling gradient-based optimisation (e.g. L-BFGS, Adam).

Occam's Razor: The marginal likelihood automatically balances data fit against model complexity - prefer the simplest model that explains the data well.

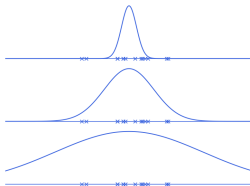
Example: Fitting the Length Scale Parameter



The mean posterior predictive function is plotted for 3 different length scales. Notice, that an almost exact fit to the data can be achieved by reducing the length scale – but the marginal likelihood does not favour this!

An Illustrative Analogous Example

Fitting the variance σ^2 of a zero-mean Gaussian to n scalar observations:



Too narrow (top), about right (middle), too wide (bottom) - marginal likelihood selects the best σ^2 .

Cholesky Decomposition vs. Matrix Inversion

```
input:  $X$  (inputs),  $\mathbf{y}$  (targets),  $k$  (covariance function),  $\sigma_n^2$  (noise level),  
        $\mathbf{x}_*$  (test input)  
2:  $L := \text{cholesky}(K + \sigma_n^2 I)$   
    $\alpha := L^\top \backslash (L \backslash \mathbf{y})$  } predictive mean eq. (2.25)  
4:  $\bar{f}_* := \mathbf{k}_*^\top \alpha$   
    $\mathbf{v} := L \backslash \mathbf{k}_*$  } predictive variance eq. (2.26)  
6:  $\mathbb{V}[f_*] := k(\mathbf{x}_*, \mathbf{x}_*) - \mathbf{v}^\top \mathbf{v}$   
    $\log p(\mathbf{y}|X) := -\frac{1}{2} \mathbf{y}^\top \alpha - \sum_i \log L_{ii} - \frac{n}{2} \log 2\pi$  eq. (2.30)  
8: return:  $\bar{f}_*$  (mean),  $\mathbb{V}[f_*]$  (variance),  $\log p(\mathbf{y}|X)$  (log marginal likelihood)
```

The Problem: Naïve Inversion

Computing $(K + \sigma_n^2 I)^{-1}$ directly:

- ▶ $\mathcal{O}(n^3)$ and *numerically unstable*
- ▶ Requires storing the full inverse

The Solution: Cholesky

Factor: $K + \sigma_n^2 I = LL^\top$, then solve:

$$L\mathbf{v} = \mathbf{y}, \quad L^\top \alpha = \mathbf{v}$$

- ▶ Numerically *stable*, same $\mathcal{O}(n^3)$

- ▶ GP provides a predictive distribution:

$$p(y_* \mid x_*, \mathcal{D})$$

- ▶ In practice, a single prediction is often required.
- ▶ A loss function $L(y_{\text{true}}, y_{\text{guess}})$ measures the penalty of an incorrect prediction.
- ▶ Choose the prediction that minimizes the expected loss (risk):

$$\tilde{R}_L(y_{\text{guess}} \mid x_*) = \int L(y_*, y_{\text{guess}}) p(y_* \mid x_*, \mathcal{D}) dy_*$$

- ▶ Optimal prediction:

$$y_{\text{optimal}} = \arg \min_{y_{\text{guess}}} \tilde{R}_L(y_{\text{guess}} \mid x_*)$$